

HealthPro — APK Bug Fix Round 3

Повне завдання для Replit Agent: тест APK + нові функції + динамічний контент

Версія: 2.0 | Дата: 30 квітня 2026 р. | SQLite: ОК, дані зберігаються, APK встановлено

ЗВЕДЕНА ТАБЛИЦЯ ВСІХ ЗАВДАНЬ

№	Завдання	Модуль	Пріоритет
1	Хардкод isRu?: перенести ВСІ рядки у словники t()	features/analytics, pressure, meds	КРИТИЧНО
2	Сповіщення: LocalNotifications замість Web API	settings/notifications.js, platform.js	КРИТИЧНО
3	PDF/CSV: Filesystem+Share замість тегу a download	export/csv.js, pdf.js, platform.js	КРИТИЧНО
4	Кнопка Назад: App.minimizeApp() через @capacitor/app	src/app.js або pwa/index.js	КРИТИЧНО
5	Вікно дозволу сповіщень після дисклеймера (1й запуск)	settings/disclaimer.js, index.html	КРИТИЧНО
6	Вікно доступу до сховища після вікна сповіщень	platform.js, settings/disclaimer.js	КРИТИЧНО
7	Фоновий режим: AndroidManifest + TYPE_STEP_COUNTER	android/AndroidManifest.xml, steps/	ВАЖЛИВО
8	Посилання ВОЗ: відновити з еталонного WHO_INFO	pressure/who.js	ВАЖЛИВО
9	Посилання в рекомендаціях: відновити всі links: []	analytics/recommendations.js	ВАЖЛИВО
10	Видалити застарілі блоки та попередження з UI	index.html, i18n словники	ВАЖЛИВО
11	Email/SMS нагадування через mailto: та sms:	index.html, settings/notifications.js	ВАЖЛИВО
12	Іконка APK: логотип HealthPro замість дефолтної	android/res/ + capacitor assets	ВАЖЛИВО

1 3	Крокомір: TYPE_STEP_COUNTER замість акселерометра	features/steps/index.js	ВАЖЛИВО
1 4	Аудит бази ліків: warn-тексти зрозумілою мовою	meds/drug-db.js	ДРІБНЕ
1 5	РОЗРОБКА: Динамічні посилання через SQLite + фон	features/content/ (новий модуль)	СПРИНТ 2

ЧАСТИНА 1 — КРИТИЧНІ ЗАВДАННЯ (1-6)

Завдання 1 — Хардкодовані рядки: системний перенос у t()

У звіті Фази 1 заявлено '179 ключів UA = 179 RU, різниця 0' — але перевірялись тільки словники між собою, не перевірялось чи ВЕСЬ текст іде через словники. Насправді десятки рядків сидять хардкодом через патерн `isRu ? '...' : '...'`. Це робить додавання третьої мови НЕМОЖЛИВИМ без переписування коду.

А

Знайти ВСІ хардкодовані рядки

- `grep -rn "isRu ?" src/features/ --include='*.js' > /tmp/hc.txt`
- `grep -rn "state.lang === 'ru'" src/features/ --include='*.js' >> /tmp/hc.txt`
- `grep -rn "lang === 'ru'" src/features/ --include='*.js' >> /tmp/hc.txt`
- `cat /tmp/hc.txt | wc -l` (зафіксувати кількість до початку роботи)
- Список підтверджених файлів: `analytics/index.js`, `pressure/who.js`, `recommendations.js`, `export/pdf.js`

Б

Алгоритм виправлення для кожного рядка

1. Знайти: `isRu ? 'рос текст' : 'укр текст'`
2. Придумати ключ: `'trend-stable'`, `'score-excellent'`, `'who-modal-title'` тощо
3. Додати ключ у `T_UK` у `src/i18n/ui.uk.js` з українським текстом
4. Додати той самий ключ у `T_RU` у `src/i18n/ui.ru.js` з російським текстом
5. Замінити `isRu ? '...' : '...'` на `t('ключ')` у коді
6. Перевірити обидві мови у браузері

В

Критерій завершення Завдання 1

- `grep -rn "isRu ?" src/features/ --include='*.js'` повертає 0 результатів
- `grep -rn "state.lang === 'ru'" src/features/ --include='*.js'` повертає 0
- Перемикання UA/RU перекладає абсолютно все без виключень
- `npm test` → 41/41 зелений

Завдання 2 — Нативні сповіщення через LocalNotifications

АРК показує 'Браузер не підтримує сповіщення' бо використовується Web Notification API. Плагін `@capacitor/local-notifications` вже встановлений — треба його підключити.

А

Оновити platform.js — замінити Web API на Capacitor

- import { LocalNotifications } from '@capacitor/local-notifications'
- requestNotificationPermission():
- if (isNative()) return (await LocalNotifications.requestPermissions()).display === 'granted'
- else return Notification.requestPermission() === 'granted'
- notify(title, body, id, triggerAt):
- if (isNative()) await LocalNotifications.schedule({
- notifications: [{ title, body, id: id || Date.now(),
- schedule: triggerAt ? { at: new Date(triggerAt) } : undefined }
- })
- else if (triggerAt) setTimeout(() => new Notification(title,{body}), triggerAt-Date.now())
- cancelNotification(id): if(isNative()) LocalNotifications.cancel({notifications:[{id}]})

Б

Оновити notifications.js — використати platform.js

- Замінити new Notification() → platform.notify()
- Замінити Notification.requestPermission() → platform.requestNotificationPermission()
- При збереженні часу нагадування → планувати через platform.notify з triggerAt
- Ранкове нагадування: platform.notify('HealthPro', текст, 'morning', тригер_час_ранку)
- Вечірнє нагадування: platform.notify('HealthPro', текст, 'evening', тригер_час_вечора)
- При прийомі ліків: планувати для кожного препарату окремо

В

Критерій завершення Завдання 2

- Нагадування про ліки спрацьовує у призначений час на APK
- Нагадування спрацьовує при ВИМКНЕНОМУ додатку
- Жодного тексту 'Браузер не підтримує' у нативному APK

Завдання 3 — PDF/CSV збереження на Android

Файли генеруються але не з'являються у файловому менеджері. Тег [a download] та jsPDF.save() у Android WebView кладуть файл у непублічний кеш. Треба: Filesystem.writeFile → Directory.Documents → Share.share.

А

Оновити platform.download() у platform.js

- import { Filesystem, Directory } from '@capacitor/filesystem'
- import { Share } from '@capacitor/share'
- async download(filename, blob, mimeType) {
- if (isNative()) {
- const base64 = await new Promise(res => {
- const r = new FileReader()
- r.onload = () => res(r.result.split(',')[1])
- r.readAsDataURL(blob)
- })
- const result = await Filesystem.writeFile({
- path: filename, data: base64,
- directory: Directory.Documents,
- recursive: true
- })
- await Share.share({ title: filename, url: result.uri, dialogTitle: 'Зберегти або поділитись' })
- } else {
- // web fallback: a.download як раніше
- const url = URL.createObjectURL(blob)
- const a = document.createElement('a')
- a.href = url; a.download = filename; a.click()
- setTimeout(() => URL.revokeObjectURL(url), 1000)
- }
- }

Б

Оновити csv.js — використати platform.download

- import { download } from '../core/platform.js'
- Замінити createElement('a') + click() на:
- const blob = new Blob([csvContent], { type: 'text/csv;charset=utf-8' })
- await download(filename + '.csv', blob, 'text/csv')
- Після завантаження: модалка закривається автоматично (вже є у обгортці)

В

Оновити pdf.js — використати platform.download

- import { download } from '../core/platform.js'
- Замінити doc.save(filename) на:
- const blob = doc.output('blob')
- await download(filename + '.pdf', blob, 'application/pdf')

Г

Критерій завершення Завдання 3

- CSV файл з'являється у файловому менеджері Android → папка Документи
- PDF файл з'являється у файловому менеджері Android → папка Документи
- Після генерації відкривається системний Share Sheet
- На ПК у браузері — файл завантажується як раніше (web fallback)

Завдання 4 — Кнопка Назад: згорання замість ігнорування

При натисканні апаратної кнопки Назад на Android нічого не відбувається. Закрити додаток можна тільки через таск-менеджер. Треба обробити подію через `@capacitor/app`.

А

Підключити обробник `backButton` у `app.js`

- `import { App } from '@capacitor/app'`
- `// Додати при ініціалізації:`
- `if (platform.isNative()) {`
- `App.addListener('backButton', ({ canGoBack }) => {`
- `if (canGoBack) {`
- `window.history.back() // якщо є попередня сторінка`
- `} else {`
- `App.minimizeApp() // згортає додаток, не закриває`
- `}`
- `}}`
- `}`

Б

Критерій завершення Завдання 4

- Кнопка Назад згортає додаток (іконка залишається в таск-менеджері)
- При повторному відкритті з таск-менеджера — стан збережений
- Подвійне натискання Назад — мінімізує (не виходить повністю)

Завдання 5 — Діалог дозволу сповіщень після дисклеймера

При першому запуску після прийняття дисклеймера показати кастомне модальне вікно з запитом дозволу на сповіщення. Мова вікна — та що обрана при дисклеймері. Це відкриває системний діалог Android для дозволу push-сповіщень.

А

Логіка показу після `acceptDisclaimer()`

- В `settings/disclaimer.js` після `acceptDisclaimer()`:
- `const asked = await storage.get('notif_permission_asked')`
- `if (!asked) { showNotificationPermissionDialog() }`
-
- Функція `showNotificationPermissionDialog()`:
- Показати кастомне модальне вікно (не системне) з текстами через `t()`:
- Заголовок UA: 'Дозволити сповіщення?'
- Заголовок RU: 'Разрешить уведомления?'
- Опис UA: 'Необхідно для нагадувань про прийом ліків та вимір тиску'
- Опис RU: 'Необходимо для напоминаний о приёме лекарств и измерении давления'
- Кнопки: Так / Ні

Б	При натисканні Так • <code>const granted = await platform.requestNotificationPermission()</code> • <code>// Це відкриє системний діалог Android для дозволу push</code> • <code>await storage.set('notif_permission_asked', 'true')</code> • <code>await storage.set('notif_permission_granted', granted ? 'true' : 'false')</code> • <code>// Перейти до наступного діалогу — сховище (Завдання 6)</code>
В	При натисканні Ні • <code>await storage.set('notif_permission_asked', 'true')</code> • <code>await storage.set('notif_permission_granted', 'false')</code> • Показати попередження: 'Без сповіщень нагадування про ліки не працюватимуть' • Перейти до діалогу сховища (Завдання 6)
Г	Додати ключі до словників i18n • <code>'notif-dialog-title': UA: 'Дозволити сповіщення?' RU: 'Разрешить уведомления?'</code> • <code>'notif-dialog-desc': UA: 'Необхідно для нагадувань...' RU: 'Необходимо для...'</code> • <code>'notif-dialog-yes': UA: 'Так' RU: 'Да'</code> • <code>'notif-dialog-no': UA: 'Ні' RU: 'Нет'</code> • <code>'notif-dialog-warn': UA: 'Без сповіщень нагадування не працюватимуть'</code>

Завдання 6 — Діалог доступу до локального сховища

Відразу після вікна сповіщень. Пояснити що зберігається на пристрої. Дає Android-дозвіл на доступ до папок для збереження звітів PDF/CSV.

А	Кастомне модальне вікно • Показати відразу після закриття діалогу сповіщень • Заголовок UA: 'Дозволити зберігати дані на пристрої?' • Заголовок RU: 'Разрешить сохранять данные на устройстве?' • Перелік що зберігається: звіти для лікаря (PDF/CSV), журнал вимірювань, налаштування • Кнопки: Дозволити / Пізніше
Б	При натисканні Дозволити • <code>if (platform.isNative()) {</code> • <code>const perm = await FileSystem.requestPermissions()</code> • <code>// Відкриває системний діалог Android дозволу до файлів</code> • <code>}</code> • <code>await storage.set('storage_permission_asked', 'true')</code> • Перейти до основного екрану додатку

В

Критерій завершення Завдань 5 та 6

- При першому запуску після дисклеймера — два діалоги послідовно
- При повторних запусках — діалоги НЕ показуються
- Мова діалогів відповідає мові обраній на екрані дисклеймера
- PDF/CSV зберігаються у папці Документи після надання дозволу

ЧАСТИНА 2 — ВАЖЛИВІ ЗАВДАННЯ (7-14)

Завдання 7 — Фоновий режим та TYPE_STEP_COUNTER

Таск-менеджер Android вбиває процес — страждають крокомір і сповіщення. Правильне рішення для кроків: замість акселерометра (поріг 11.5 — занадто великий) використати системний сенсор TYPE_STEP_COUNTER.

А

AndroidManifest.xml — додати дозволи

- Відкрити android/app/src/main/AndroidManifest.xml
- Додати у блок manifest:
- uses-permission: android.permission.FOREGROUND_SERVICE
- uses-permission: android.permission.RECEIVE_BOOT_COMPLETED
- uses-permission: android.permission.WAKE_LOCK
- uses-feature: android.hardware.sensor.stepcounter (required=false)

Б

Замінити акселерометр на TYPE_STEP_COUNTER у steps/index.js

- TYPE_STEP_COUNTER — вбудований сенсор Android, точний, не потребує Foreground Service
- Він накопичує кроки з моменту завантаження телефону
- Алгоритм:
- 1. При першому старті дня: зберегти baseline = поточне значення сенсора
- 2. Кроки сьогодні = поточне значення - baseline
- 3. Скидати baseline щодня о 00:00
- Через @capacitor/motion або Capacitor Plugin API:
- Motion.addListener('accel', ...) → замінити на подію сенсора кроків
- Якщо TYPE_STEP_COUNTER недоступний — знизити поріг акселерометра до 2.5

Завдання 8 — Відновити посилання ВОЗ з еталонного WHO_INFO

Файл Посилання.txt містить еталонний WHO_INFO зі старого коду. Треба звірити поточний who.js з цим еталоном і відновити втрачені посилання.

А

Еталонні посилання з WHO_INFO (відновити всі)

- optimal: МОЗ 8 міфів (moz.gov.ua) + МОЗ правила виміру (moz.gov.ua)
- normal: МОЗ правила виміру + Гіпертензія ЦГЗУ (phc.org.ua)
- high-normal: МОЗ 8 міфів + ЦГЗУ артеріальна гіпертензія
- ht1: МОЗ 8 міфів + ВООЗ гіпертонія (who.int) + Helsi (helsi.me)
- ht2: ВООЗ гіпертонія + ЦГЗУ + Helsi
- ht3: Виклик 103 (tel:103) + ВООЗ + Helsi

Б

Звірити поточний who.js з еталоном

- Відкрити src/features/pressure/who.js
- Порівняти links: [] для кожної категорії з еталоном вище
- Додати відсутні посилання
- Переконайтесь що посилання відкриваються через platform.openUrl(url)

Завдання 9 — Відновити посилання в рекомендаціях

Частина посилань у блоці рекомендацій зникла. Для кожної рекомендації перевірити масив links і відновити.

А

Перевірити та відновити у recommendations.js

- Відкрити src/features/analytics/recommendations.js
- Для кожної рекомендації перевірити: чи є links: [] і чи не порожній
- Харчування та DASH-дієта: МОЗ 'Що і як їсти' (moz.gov.ua)
- Фізична активність: ВООЗ фізична активність (who.int)
- Сон та управління стресом: МОЗ ментальне здоров'я (moz.gov.ua)
- Відмова від куріння: ЦГЗУ тютюнопаління (phc.org.ua)
- Всі посилання відкривати через platform.openUrl(url)

Завдання 10 — Видалити застарілі блоки та попередження

Кілька блоків у додатку містять застарілі технічні пояснення які заплутують користувача або вже неактуальні.

А

Список що видалити або замінити

1. Блок Нагадування: видалити текст 'Нагадування активні лише коли додаток відкритий'
- Замінити на: 'Нагадування працюють навіть при вимкненому додатку'
2. Видалити текст 'Після встановлення через PWABuilder фонові push потребують VAPID-сервера'
3. Блок Email/SMS: видалити 'Автоматична відправка Email потребує серверної частини'
- Замінити на реальний UI з кнопками (Завдання 11)
4. Перевірити інші місця де є технічні пояснення для розробника замість тексту для користувача
- 5. `grep -rn 'PWABuilder\|VAPID\|серверн' src/i18n/ --include='*.js'`

Завдання 11 — Email/SMS нагадування через mailto: та sms:

Реалізувати без бекенду через нативні протоколи. Відкривають нативний поштовий клієнт або SMS на Android. Дані беруться з профілю користувача.

А	Реалізація Email через mailto: • <code>const email = state.settings.userEmail</code> • <code>const subj = encodeURIComponent(t('notif-email-subject'))</code> • <code>const body = encodeURIComponent(t('notif-email-body') + ' ' + new Date().toLocaleString())</code> • <code>const mailto = 'mailto:' + email + '?subject=' + subj + '&body;=' + body</code> • <code>platform.openUrl(mailto)</code> // відкриє нативний поштовий клієнт
Б	Реалізація SMS через sms: • <code>const phone = state.settings.userPhone</code> • <code>const smsText = encodeURIComponent(t('notif-sms-body'))</code> • <code>const sms = 'sms:' + phone + '?body=' + smsText</code> • <code>platform.openUrl(sms)</code> // відкриє нативний SMS-клієнт
В	UI для блоку Email/SMS у налаштуваннях • Показати email та телефон з профілю (або підказку заповнити профіль) • Чекбокси: 'Нагадування про ліки', 'Ранкове нагадування', 'Вечірнє нагадування' • Кнопки: 'Надіслати Email' та 'Надіслати SMS' • Якщо email/телефон не заповнені — показати підказку перейти у Профіль • Всі тексти через i18n словники
Г	Додати ключі до словників i18n • 'notif-email-subject': UA: 'HealthPro: нагадування' RU: 'HealthPro: напоминание' • 'notif-email-body': UA: 'Час виміряти тиск!' RU: 'Время измерить давление!' • 'notif-sms-body': UA: 'HealthPro: час виміряти тиск!' RU: 'HealthPro: время измерить давление!' • 'notif-send-email': UA: 'Надіслати Email' RU: 'Отправит Email' • 'notif-send-sms': UA: 'Надіслати SMS' RU: 'Отправит SMS'

Завдання 12 — Іконка APK: логотип HealthPro

А	Згенерувати через @capacitor/assets • Підготувати icon.png 1024x1024px (логотип HealthPro з хвилею серцебиття) • Покласти у resources/icon.png у корені проекту • <code>npm install -g @capacitor/assets</code> (якщо не встановлено) • <code>npm capacitor-assets generate --android</code> • <code>npm cap sync android</code> • Це автоматично створить всі розміри: hdpi, xhdpi, xxhdpi, xxxhdpi
---	--

Завдання 13 — Крокомір: точний підрахунок

A

Замінити акселерометр на TYPE_STEP_COUNTER

- Поточне STEP_ACCEL_THRESHOLD = 11.5 — занадто високий поріг для ходьби
- Детальна реалізація описана у Завданні 7-Б
- Як тимчасовий фікс: знизити поріг до 2.5-3.0 у constants.js
- Фінальне рішення: TYPE_STEP_COUNTER через Android Sensor API

Завдання 14 — Аудит бази ліків: warn-тексти

A

Перевірити та покращити warn-тексти

- Відкрити src/features/meds/drug-db.js
- 'не натщесерце' → 'приймати після їжі' (зрозуміліше для пацієнта)
- 'контроль K+' → 'контролювати рівень калію в крові (аналіз)',
- 'контроль МНВ' → 'регулярний аналіз крові на МНВ (протромбіновий час)'
- Перевірити що кожен препарат має warn в обох мовах (UA та RU ключі)
- Провести повний аудит всіх 50+ препаратів

ЧАСТИНА 3 — РОЗРОБКА: ДИНАМІЧНІ ПОСИЛАННЯ (СПРИНТ 2)

Важливо: це окремий спринт після стабілізації Round 3

Не починати поки всі 14 завдань Round 3 не завершені та APK не стабільний. Динамічні посилання — це нова функція, а не баг-фікс.

Ідея: посилання в рекомендаціях поділяються на два типи. Статичні — офіційні джерела МОЗ, ВООЗ, ЦГЗУ — не змінюються, зашиті в код. Динамічні — тематичні статті, новини здоров'я — підтягуються у фоні через SQLite і оновлюються без оновлення APK.

Архітектура нового модуля `src/features/content/`

1

Схема таблиці `dynamic_links` у SQLite

- `CREATE TABLE IF NOT EXISTS dynamic_links (`
- `id INTEGER PRIMARY KEY AUTOINCREMENT,`
- `category TEXT NOT NULL, -- 'hypertension', 'nutrition', 'activity', 'stress'`
- `title_uk TEXT NOT NULL,`
- `title_ru TEXT NOT NULL,`
- `url TEXT NOT NULL,`
- `source TEXT, -- 'moz', 'who', 'phc', 'other'`
- `is_static INTEGER DEFAULT 0, -- 1 = не видаляти при оновленні`
- `fetches_at INTEGER, -- timestamp останнього оновлення`
- `expires_at INTEGER -- коли вважати застарілим`
- `)`

2

Файли нового модуля `src/features/content/`

- `index.js` — публічний API: `getLinks(category)`, `refreshLinks()`, `init()`
- `fetcher.js` — фонове завантаження посилань з джерел
- `sources.js` — список джерел для кожної категорії (RSS, API)
- `parser.js` — парсинг RSS/JSON відповідей
- `storage.js` — запис та читання з SQLite таблиці `dynamic_links`

3

Логіка фонового оновлення

- При запуску додатку: `content.init()`
- Перевірити `expires_at` для кожної категорії
- Якщо застарілі (> 24 год) → запустити `refreshLinks()` у фоні
- `refreshLinks()` не блокує UI — використати `setTimeout` або `Web Worker`
- Нові посилання записати у SQLite через `db.run(INSERT OR REPLACE)`
- Статичні посилання (`is_static=1`) ніколи не видаляти

4

Джерела для кожної категорії

- hypertension: RSS МОЗ → moz.gov.ua/rss або phc.org.ua/rss
- nutrition: RSS BOOЗ або МОЗ харчування
- activity: BOOЗ фізична активність
- stress/sleep: МОЗ ментальне здоров'я
- Якщо RSS недоступний → залишити статичні посилання
- Fallback: завжди є мінімум 1 статичне посилання на категорію

5

Інтерпація з recommendations.js

- Замінити статичний масив links: [] на:
- `const links = await content.getLinks(reco.category)`
- `getLinks()` повертає: спочатку статичні, потім динамічні (за датою)
- Якщо динамічних немає — тільки статичні
- Максимум 4-5 посилань на рекомендацію щоб не захарашувати UI

6

Критерій завершення Спринту 2

- При відкритті рекомендацій: посилання завантажуються миттєво (з SQLite)
- У фоні: посилання оновлюються кожні 24 год без гальмування UI
- Статичні МОЗ/BOOЗ посилання завжди присутні (`is_static=1`)
- При відсутності інтернету — тільки статичні (`graceful degradation`)
- `npm test` → всі тести зелені включно з новими для content/

РЕКОМЕНДОВАНИЙ ПОРЯДОК ВИКОНАННЯ

Кр ок	Дія	Час	Пріоритет
1	Завд.1: grep хардкоду → перенести BCI isRu? у словники t()	4 год	СПОЧАТКУ
2	Завд.8: who.js — відновити посилання з еталонного WHO_INFO	1 год	СПОЧАТКУ
3	Завд.4: App.addListener(backButton) + minimizeApp()	30 хв	СПОЧАТКУ
4	Завд.2: LocalNotifications замість Web Notification API	2 год	СПОЧАТКУ
5	Завд.3: platform.download + Filesystem + Share	2 год	СПОЧАТКУ
6	Завд.5: діалог дозволу сповіщень після дисклеймера	1 год	СПОЧАТКУ
7	Завд.6: діалог доступу до сховища після сповіщень	1 год	СПОЧАТКУ
8	npm test → 41/41, git push, CI зелений, новий APK	15 хв	СПОЧАТКУ
9	Завд.7: AndroidManifest дозволи + TYPE_STEP_COUNTER	2 год	Потім
10	Завд.9: посилання в рекомендаціях — відновити links: []	1 год	Потім
11	Завд.10: видалити застарілі блоки PWABuilder/VAPID/бекенд	1 год	Потім
12	Завд.11: Email/SMS через mailto: та sms: протоколи	2 год	Потім
13	Завд.12: іконка APK через capacitor-assets generate	30 хв	Потім
14	Завд.13: крокомір — TYPE_STEP_COUNTER або поріг 2.5	2 год	Потім
15	Завд.14: аудит бази ліків — warn-тексти	1 год	Наприкінці
16	Фінальний APK тест: всі 14 пунктів на реальному пристрої	1-2 год	ОБОВ'ЯЗКОВО
17	СПРИНТ 2: динамічні посилання — тільки після стабільного APK	3-5 днів	СПРИНТ 2

ВАЖЛИВІ ЗАСТЕРЕЖЕННЯ

Хардкод isRu? — системна проблема, не точкова

Після виправлення: grep -rn 'isRu ?' src/features/ має повернути рівно 0 результатів. Не виправляти по одному — знайти BCI через grep і виправити системно за один підхід.

npm test після КОЖНОГО завдання — 41/41

Після кожного окремого завдання запускати npm test. Якщо тест впав — зупинитись і виправити одразу, не рухатись далі з червоним тестом.

Новий APK після кроків 1-8 (критичних)

git push → GitHub Actions збирає APK автоматично → завантажити з Artifacts → встановити на пристрій → підтвердити що сповіщення, PDF/CSV та кнопка Назад працюють.

Динамічні посилання — тільки після стабільного APK

Спринт 2 починати тільки після того як всі 14 завдань Round 3 виконані і APK протестований на реальному пристрої без критичних багів.

Email/SMS без бекенду — правильне рішення

mailto: та sms: протоколи — стандартний підхід для мобільних додатків. Відкривають нативний поштовий клієнт або SMS. Не потребують сервера, GDPR, SSL або додаткових дозволів.

Файл Посилання.txt — еталон для who.js

Файл містить повний WHO_INFO зі старого коду з усіма посиланнями. Використати як еталон при відновленні посилань у Завданні 8.